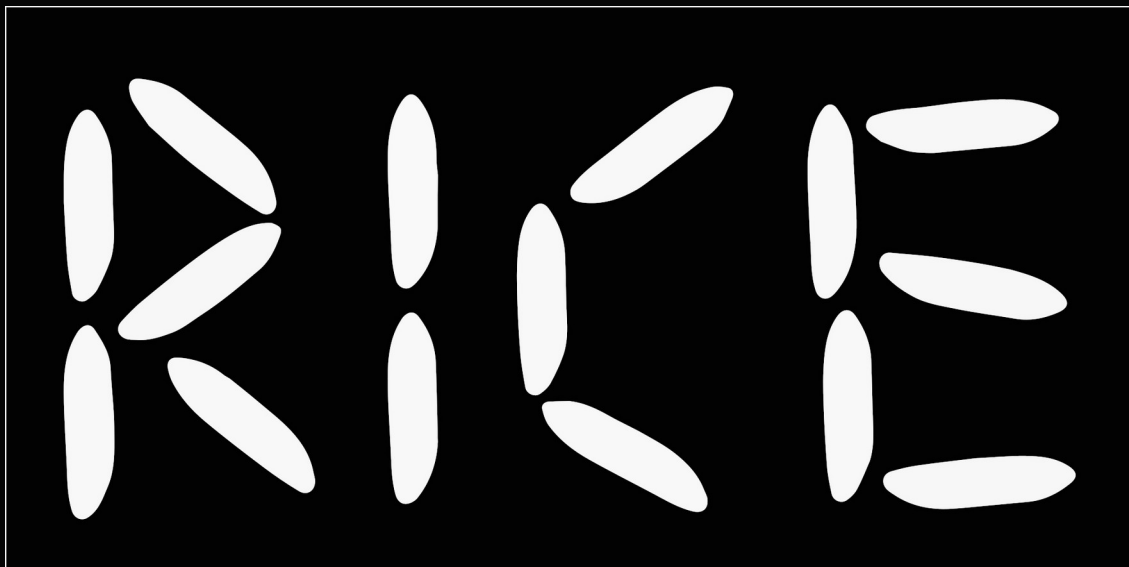




SAR simulation environment

User Manual

Version 40 - Bilingual edition

Authors: Zoe Betta and Antonio Sgorbissa

Contents

1. Purpose and scope
2. Installation and first launch
3. Quick start
4. Experiment configuration
5. Designing experiments
6. Running and observing the simulation
7. Output files
8. Metrics and interpretation
9. Statistical analysis workbook
10. Reproducibility checklist
11. Troubleshooting
12. File structure

1. Purpose and scope

SAR simulation environment is a Python application for evaluating multi-floor Search and Rescue exploration strategies with a simulated quadruped robot. It generates repeatable buildings, runs all requested Fw and Opt conditions on the same environment and initial pose, displays the exploration process, and exports descriptive and inferential statistics.

Core principle Within one building, every Fw x Opt condition uses the same geometry and validated starting pose. This supports fair and reproducible comparisons.

2. Installation and first launch

2.1 Requirements

- Python 3.10 or later.
- A desktop environment for Tkinter, Pygame, Matplotlib, and the optional Panda3D window.
- A GPU is recommended, but not required, for the optional 3D view.

2.2 Install the dependencies

Open a terminal in the distribution folder. A virtual environment is recommended:

```
python -m venv .venv
.venv\Scripts\activate # Windows
source .venv/bin/activate # Linux/macOS
python -m pip install -r requirements.txt
```

2.3 Start the program

```
python main.py
```

The welcome screen shows the SAR logo, the application title, the authors, and a language selector. Choose English or Italiano and select Continue. The choice applies to the configuration window, live viewer, result windows, and console messages. Open manual opens the PDF matching the selected language.

3. Quick start

1. Run `python main.py`.
2. Select English and press Continue.
3. For a quick test, set Number of random buildings to 1 and Run duration Tr to 60 s.
4. Press Start simulation.
5. Use SPACE to pause, keys 1-4 to change speed, and Max Turbo for the fastest execution.
6. Review the final tables and the generated CSV and XLSX files.

4. Experiment configuration

The configuration window is divided into batch parameters, fixed planner weights, and environment settings. Only Fw and Opt generate a Cartesian product of conditions; all other values remain fixed throughout the batch.

4.1 Batch parameters

Option	Default	Meaning and guidance
Number of floors N	4	Number of floors in each generated building. Minimum: 2.
Run duration Tr [s]	600	Total time budget assigned to one episode.
Floor-transition cost Ts [s]	60	Time removed from the budget whenever the robot changes floor. It does not increase t_{expl} .
Number of random buildings	5	Independent building seeds. Increase this value for stable statistical comparisons.
Initial global seed	0	The first building seed. Building i uses $base_seed + i$.
Combinatorial Fw	0; 0.3; 1	One or more non-negative values separated by semicolons or commas.
Combinatorial Opt	ON; OFF	One or both optimization modes. Every Fw x Opt combination is executed.

4.2 Fixed planner weights

Parameter	Default	Role
Iw	1	Rewards expected information gain at a frontier.
Dw	1	Penalizes travel distance.
Vw	0.3	Penalizes the planner proximity term.
Cw	3	Curvature penalty used when Opt=ON; discourages unnecessarily curved paths.
Ow	8	Orientation penalty used when Opt=ON; discourages deviations from the preferred heading.
Wp	24	Frontier-persistence reward; reduces rapid target switching.
Ms	1	Minimum advantage required before switching to another target.

Experimental advice Change one parameter family at a time whenever possible. Record every configuration value, the seed, and the software version in the laboratory log.

4.3 Environment settings

Option	Choices	Meaning
Topology	Office-like floors / Free topology	Office layouts contain rooms and corridors; free topology produces less structured spaces.
Add chairs, tables, and debris	Enabled / Disabled	Adds physical obstacles and corresponding 2D/3D visual objects.
Object density	1-4	Scales obstacle quantity. Higher values make navigation and generation more demanding.

5. Designing experiments

5.1 Episode count

Total episodes = number of buildings x number of Fw values x number of Opt values. For example, 170 buildings, three Fw values, and two Opt values produce 1,020 episodes.

5.2 Fair comparisons

For each building, the program validates a starting pose that works for every condition. The building and starting pose are reused for the complete Fw x Opt batch. If the robot cannot plan a path or translate during the diagnostic interval, a new pose is selected before any episode for that building is accepted.

5.3 Suggested scales

Purpose	Suggested setup
Interface check	1 building, Tr=30-60 s, one Fw value, one Opt value.
Pilot comparison	10-30 buildings with all intended conditions.
Publication-scale experiment	Use enough buildings for stable distributions; keep identical buildings across conditions and report the full design.

6. Running and observing the simulation

Control	Action
SPACE	Pause or resume the current episode.
1 / 2 / 3 / 4	Set playback speed to 1x, 2x, 4x, or 8x.
Max Turbo	Freeze rendering and execute fixed simulation steps as quickly as possible.

Control	Action
Enable 3D view	Open the Panda3D first-person camera for the current building.
ESC	End the current episode.

The left panel shows continuous ground-truth geometry. The right panel shows the robot-built probabilistic and semantic map. The legend separates the real environment, robot and sensors, planning overlays, and internal map states.

6.1 Start diagnostics

Before a building batch starts, a short invisible run verifies that the robot can plan and translate. Progress is shown in the persistent viewer. A watchdog prevents pathological maps from leaving the program apparently frozen.

6.2 Max Turbo and 3D

Max Turbo suspends expensive rendering but does not change the deterministic physical time step. The 3D preference is preserved and the Panda3D window is restored when Turbo is stopped, if it was previously enabled.

7. Output files

File	Contents
sar_simulation_results.csv	One row per episode: configuration, coverage, floor activity, time, and SAR detection metrics.
sar_simulation_condition_statistics.csv	Means and sample SDs for Af, Af*, and Atot for each Fw/Opt condition.
sar_simulation_time_area_relationship.csv	Correlation, regression, model selection, and conclusions for t_expl versus Atot.
sar_simulation_analysis.xlsx	Raw results, summaries, ANOVA, Tukey-Kramer, Welch, Fw analyses, time-area models, and notes.

8. Metrics and interpretation

Metric	Definition	Interpretation
Af	Mean percentage explored over the floors that were visited.	How thoroughly the robot covered the floors it reached.
Af*	Percentage explored on the starting floor.	Coverage of the initial floor.
Atot	Explored area divided by total explorable area over all floors.	Primary global exploration measure.
Vf	Number of distinct floors visited.	Vertical reach.
Cf	Number of floor changes.	Frequency of vertical switching.

Metric	Definition	Interpretation
t_expl	Simulated time actually spent exploring.	Ts reduces the budget but is not added to t_expl.
TP / FP / FN / TN	Victim-detection confusion values.	Inputs for sensitivity, specificity, balanced accuracy, and MCC.

8.1 Joint interpretation

High Af with low Atot may indicate that the robot explored only a few floors very thoroughly. High Vf or Cf does not automatically imply high Atot because transitions consume Ts. The desired behavior often balances local coverage and timely floor changes.

9. Statistical analysis workbook

Sheet	How to read it
Run results	Episode-level observations for custom analyses and plots.
Condition stats	Descriptive statistics for each Fw/Opt condition.
ANOVA conditions	Omnibus ANOVA across all conditions. A significant p-value indicates that at least one condition differs.
Tukey conditions	All pairwise comparisons; use p_bonferroni or reject_0.05_bonferroni.
Welch Opt	Opt=ON versus Opt=OFF after pooling Fw; Hedges g is the effect size.
ANOVA Fw	Comparison among Fw levels after pooling Opt; eta-squared is the omnibus effect size.
Tukey Fw	Pairwise Fw comparisons with Bonferroni correction and Hedges g.
Time-area relation	Pearson, Spearman, linear slope, selected model, and conclusion for t_expl versus Atot.
Time-area models	Diagnostics for linear, quadratic, logarithmic, square-root, and saturating-exponential models.
Notes	Definitions and interpretation guidance.

9.1 Effect sizes

- ANOVA uses eta-squared: the proportion of total variance associated with group membership.
- Welch and Tukey-Kramer comparisons use Hedges g: a bias-corrected standardized difference between two means.
- Statistical significance and practical magnitude are complementary; report p-values and effect sizes together.

9.2 Time-area relationship

The software compares linear, quadratic, logarithmic, square-root, and saturating-exponential models using AICc. A nonlinear model is selected only when its advantage is strong enough; otherwise the simpler linear description is retained.

10. Reproducibility checklist

- Archive the complete distribution and requirements.txt.
- Record every configuration value and the software version.
- Preserve the initial seed and number of buildings.
- Use the same buildings for all conditions being compared.
- Copy or rename output files before running another experiment in the same folder.
- Report the metrics, statistical tests, multiple-comparison correction, and effect sizes.

11. Troubleshooting

Problem	Action
Pygame is not installed	Run <code>python -m pip install pygame-ce</code> or reinstall requirements.txt.
The 3D view does not open	Install Panda3D, update the graphics driver, and test outside remote desktop.
A building cannot be initialized	Reduce object density or try another seed; structural errors are reported explicitly.
The program is slow between buildings	The start diagnostic is validating the next building. Watch the progress strip or enable Max Turbo.
Inferential tests are not meaningful	Increase the number of buildings so each condition has repeated observations.
Files are overwritten	Move or rename CSV/XLSX outputs before the next experiment.

12. File structure

main.py orchestrates the experiment; gui.py implements the Tk, Pygame, and Matplotlib interfaces; simulation.py contains run state and metrics; building.py generates environments; planner.py selects frontiers; robot.py implements sensing and motion; camera3d.py renders the optional first-person view; statistical_analysis.py creates analyses and Excel exports; i18n.py contains translations; welcome.py implements the bilingual start screen.